

Wireshark doo dooo do doo

 **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

Can you find the flag?

Tools Used

- `file` – Determine file type
 - `strings` – Extract printable character sequences
 - `exiftool` – Read PCAP metadata
 - **Wireshark** – GUI tool for analyzing network traffic
 - `tshark` – Command-line Wireshark tool for packet analysis
 - `sort` – Organize timestamps in chronological order
 - `cut` – Extract specific fields from output
 - `xxd` – Convert hexadecimal to binary data
 - `tr` – Translate characters (ROT13 decoding)
-

Complete Solution

Step 1: Download and Prepare

Download the PCAP file named `shark1.pcapng` from the challenge page.

Step 2: Basic File Inspection

First, verify the file type and general information:

```
file shark1.pcapng
```

Expected output:

```
shark1.pcapng: pcapng capture file - version 1.0
```

The file is a **PCAPNG** (PCAP Next Generation) capture file containing network traffic.

Step 3: Search for Visible Text

Check if the flag appears as plain text:

```
strings shark1.pcapng | grep -i "picoCTF"
```

Output: (no results)

The flag isn't directly visible. We need to analyze the network traffic.

Step 4: Examine Metadata

```
exiftool shark1.pcapng
```

Output:

ExifTool Version Number	: 13.50
File Name	: shark1.pcapng
Directory	: .
File Size	: 1559 kB
File Modification Date/Time	: 2026:05:17 14:35:17+02:00
File Access Date/Time	: 2026:05:17 14:35:40+02:00
File Inode Change Date/Time	: 2026:05:17 14:35:27+02:00
File Permissions	: -rw-rw-rw-
File Type	: PCAPNG
File Type Extension	: pcapng

MIME Type	: application/vnd.tcpdump.pcap
PCAP Version	: PCAPNG 1.0
Byte Order	: Little-endian (Intel, II)
Hardware SSE4.2)	: Intel(R) Xeon(R) CPU E5-2676 v3 @ 2.40GHz (with
User Application	: Dumpcap (Wireshark) 3.2.5 (v3.2.5-0-ged20ddea8138)
Link Type	: IEEE 802.3 Ethernet
Device Name	: \Device\NPF_{EE54A6A6-8BD8-498A-995A-DE0B78012FBE}
Description	: Ethernet 4
Time Stamp Resolution	: 1e-06
Filter	: 0: not tcp port 3389
Operating System	: 64-bit Windows 10 (1909), build 18363
Time Stamp	: 2020:08:10 03:37:39.475628+02:00

The metadata doesn't reveal the flag.

Step 5: Analyze with Wireshark

Open the file in Wireshark to get a visual understanding of the traffic:

shark1pcapng

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.38.104	192.168.38.103	TCP	391	64049 → 5985 [PSH, ACK] Seq=1 Ack=1 Win=1026 Len=37 [TCP PDU reassembled in 2]
2	0.000053	192.168.38.104	192.168.38.103	HTTP	11044	POST /wsman/subscriptions/EB489718-F373-4F7F-8493-B0D1503B3C3E/37 HTTP/1.1
3	0.000315	192.168.38.103	192.168.38.104	TCP	54	5985 → 64049 [ACK] Seq=1 Ack=11328 Win=8212 Len=0
4	0.001510	192.168.38.103	192.168.38.104	TCP	1514	5985 → 64049 [ACK] Seq=1 Ack=11328 Win=8212 Len=1460 [TCP PDU reassembled in 5]
5	0.001510	192.168.38.103	192.168.38.104	HTTP	553	HTTP/1.1 200 (application/http-kerberos-session-encrypted)
6	0.001552	192.168.38.104	192.168.38.103	TCP	54	64049 → 5985 [ACK] Seq=11328 Ack=1960 Win=1026 Len=0
7	0.023972	192.168.38.104	192.168.38.103	TCP	390	64029 → 5985 [PSH, ACK] Seq=1 Ack=1 Win=8212 Len=336 [TCP PDU reassembled in 8]
8	0.023126	192.168.38.104	192.168.38.103	HTTP	7611	POST /wsman/subscriptions/EB489718-F373-4F7F-8493-B0D1503B3C3E/36 HTTP/1.1
9	0.023394	192.168.38.103	192.168.38.104	TCP	54	5985 → 64029 [ACK] Seq=1 Ack=7894 Win=8212 Len=0
10	0.024317	192.168.38.103	192.168.38.104	TCP	1514	5985 → 64029 [ACK] Seq=1 Ack=7894 Win=8212 Len=1460 [TCP PDU reassembled in 11]
11	0.024317	192.168.38.103	192.168.38.104	HTTP	553	HTTP/1.1 200 (application/http-kerberos-session-encrypted)
12	0.024348	192.168.38.104	192.168.38.103	TCP	54	64029 → 5985 [ACK] Seq=7894 Ack=1960 Win=8212 Len=0
13	0.028524	192.168.38.104	192.168.38.103	TCP	390	64029 → 5985 [PSH, ACK] Seq=7894 Ack=1960 Win=8212 Len=336 [TCP PDU reassembled in 14]
14	0.028500	192.168.38.104	192.168.38.103	HTTP	7611	POST /wsman/subscriptions/EB489718-F373-4F7F-8493-B0D1503B3C3E/36 HTTP/1.1
15	0.028767	192.168.38.103	192.168.38.104	TCP	54	5985 → 64029 [ACK] Seq=1960 Ack=15787 Win=8212 Len=0
16	0.029663	192.168.38.103	192.168.38.104	TCP	1514	5985 → 64029 [ACK] Seq=1960 Ack=15787 Win=8212 Len=1460 [TCP PDU reassembled in 17]
17	0.029663	192.168.38.103	192.168.38.104	HTTP	553	HTTP/1.1 200 (application/http-kerberos-session-encrypted)
18	0.029690	192.168.38.104	192.168.38.103	TCP	54	64029 → 5985 [ACK] Seq=15787 Ack=3919 Win=8212 Len=0
19	0.030608	192.168.38.104	192.168.38.103	TCP	390	64029 → 5985 [PSH, ACK] Seq=15787 Ack=3919 Win=8212 Len=336 [TCP PDU reassembled in 20]
20	0.030608	192.168.38.104	192.168.38.103	HTTP	7615	POST /wsman/subscriptions/EB489718-F373-4F7F-8493-B0D1503B3C3E/36 HTTP/1.1

UTC Arrival Time: Aug 10, 2020 01:51:39.797917000 UTC
Epoch Arrival Time: 1597024299.797917000
[Time shift for this packet: 0.000000000 seconds]
[Time since reference or first frame: 0.000000000 seconds]
Frame Number: 1
Frame Length: 391 bytes (3128 bits)
Capture Length: 391 bytes (3128 bits)
[Frame is marked: False]
[Frame is ignored: False]
[Protocols in frame: eth:ethertype:ip:tcp]
Character encoding: ASCII (0)
[Coloring Rule Name: TCP]
[Coloring Rule String: tcp]

Ethernet II, Src: MS-NLB-PhysServer-32_1b:c6:1a:ae:f5 (02:3b:c6:1a:ae:f5), Dst: 02:cb:a2:67:5d:0f (02:3b:c6:1a:ae:f5)
Internet Protocol Version 4, Src: 192.168.38.104, Dst: 192.168.38.103
Transmission Control Protocol, Src Port: 64049, Dst Port: 5985, Seq: 1, Ack: 1, Len: 337
Source Port: 64049
Destination Port: 5985
[Stream index: 0]
[Stream Packet Number: 1]
[Conversation completeness: Incomplete (12)]

Destination Port (tcp.dstport), 2 byte(s)

Packets: 987

Profile: Default

shark1pcapng

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

tcp.stream eq 0

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.38.104	192.168.38.103	TCP	391	64049 → 5985 [PSH, ACK] Seq=1 Ack=1 Win=1026 Len=37 [TCP PDU reassembled in 2]
2	0.000053	192.168.38.104	192.168.38.103	HTTP	11044	POST /wsman/subscriptions/EB489718-F373-4F7F-8493-B0D1503B3C3E/37 HTTP/1.1
3	0.000315	192.168.38.103	192.168.38.104	TCP	54	5985 → 64049 [ACK] Seq=1 Ack=11328 Win=8212 Len=0
4	0.001510	192.168.38.103	192.168.38.104	TCP	1514	5985 → 64049 [ACK] Seq=1 Ack=11328 Win=8212 Len=1460 [TCP PDU reassembled in 5]
5	0.001510	192.168.38.103	192.168.38.104	HTTP	553	HTTP/1.1 200 (application/http-kerberos-session-encrypted)
6	0.001552	192.168.38.104	192.168.38.103	TCP	54	64049 → 5985 [ACK] Seq=11328 Ack=1960 Win=1026 Len=0

Encapsulation type: Ethernet (1)
Arrival Time: Aug 10, 2020 03:51:39.797917000 SAST
UTC Arrival Time: Aug 10, 2020 01:51:39.797917000 UTC
Epoch Arrival Time: 1597024299.797917000
[Time shift for this packet: 0.000000000 seconds]
[Time since reference or first frame: 0.000000000 seconds]
Frame Number: 1
Frame Length: 391 bytes (3128 bits)
Capture Length: 391 bytes (3128 bits)
[Frame is marked: False]
[Frame is ignored: False]
[Protocols in frame: eth:ethertype:ip:tcp]
Character encoding: ASCII (0)
[Coloring Rule Name: TCP]
[Coloring Rule String: tcp]

Ethernet II, Src: MS-NLB-PhysServer-32_1b:c6:1a:ae:f5 (02:3b:c6:1a:ae:f5), Dst: 02:cb:a2:67:5d:0f (02:3b:c6:1a:ae:f5)
Internet Protocol Version 4, Src: 192.168.38.104, Dst: 192.168.38.103
Transmission Control Protocol, Src Port: 64049, Dst Port: 5985, Seq: 1, Ack: 1, Len: 337
Source Port: 64049
Destination Port: 5985
[Stream index: 0]

Destination Port (tcp.dstport), 2 byte(s)

Packets: 987 - Displayed: 6 (0.6%)

Profile: Default

shark1pcapng

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

tcp.stream eq 5

No.	Time	Source	Destination	Protocol	Length	Info
519	7.137396	192.168.38.104	18.222.37.134	TCP	60	54093 → 80 [SWN] Seq=0 Win=62727 Len=0 MSS=8961 WS=256 SACK_PERM
821	7.186705	192.168.38.104	192.168.38.104	TCP	66	80 → 64093 [SYN, ACK] Seq=0 Ack=1 Win=62727 Len=0 MSS=1460 SACK_PERM WS=128
822	7.186777	192.168.38.104	18.222.37.134	TCP	54	64093 → 80 [ACK] Seq=1 Ack=1 Win=262656 Len=0
823	7.187055	192.168.38.104	18.222.37.134	HTTP	501	GET / HTTP/1.1
826	7.236249	18.222.37.134	192.168.38.104	TCP	54	80 → 64093 [ACK] Seq=1 Ack=448 Win=62336 Len=0
827	7.236537	18.222.37.134	192.168.38.104	HTTP	384	HTTP/1.1 200 OK (text/html)
828	7.276719	192.168.38.104	18.222.37.134	TCP	54	64093 → 80 [ACK] Seq=448 Ack=331 Win=262400 Len=0
872	12.240842	18.222.37.134	192.168.38.104	TCP	54	80 → 64093 [FIN, ACK] Seq=331 Ack=448 Win=62336 Len=0
873	12.240874	192.168.38.104	18.222.37.134	TCP	54	64093 → 80 [ACK] Seq=448 Ack=332 Win=262400 Len=0

Arrival Time: Aug 10, 2020 03:51:46.935313000 SAST
UTC Arrival Time: Aug 10, 2020 01:51:46.935313000 UTC
Epoch Arrival Time: 1597024306.935313000
[Time shift for this packet: 0.000000000 seconds]
[Time delta from previous captured frame: 345.985000 milliseconds]
[Time since reference or first frame: 7.137396000 seconds]
Frame Number: 819
Frame Length: 66 bytes (528 bits)
Capture Length: 66 bytes (528 bits)
[Frame is marked: False]
[Frame is ignored: False]
[Protocols in frame: eth:ethertype:ip:tcp]
Character encoding: ASCII (0)
[Coloring Rule Name: HTTP]
[Coloring Rule String: http || tcp.port == 80 || http2]

Ethernet II, Src: MS-NLB-PhysServer-32_1b:c6:1a:ae:f5 (02:3b:c6:1a:ae:f5), Dst: 02:fb:68:4c:e9:41 (02:3b:c6:1a:ae:f5)
Internet Protocol Version 4, Src: 192.168.38.104, Dst: 18.222.37.134
Transmission Control Protocol, Src Port: 64093, Dst Port: 80, Seq: 0, Len: 0
Source Port: 64093
Destination Port: 80
[Stream index: 5]

Destination Port (tcp.dstport), 2 byte(s)

Packets: 987 - Displayed: 9 (0.9%)

Profile: Default

After thorough investigation, a specific TCP stream catches our attention: `tcp.stream eq 5` .

Step 6: Extract the TCP Stream with tshark

Instead of manually extracting packets, we'll use `tshark` for automated extraction:

```
tshark -nr shark1.pcapng -Y "tcp && tcp.stream eq 5" -T fields -e frame.time -e tcp.payload
```

Output:

```
2020-08-10T03:51:46.935313000+0200
2020-08-10T03:51:46.984622000+0200
2020-08-10T03:51:46.984694000+0200
2020-08-10T03:51:46.984972000+0200
474554202f20485454502f312e310d0a486f73743a2031382e3232322e33372e3133340d0a436f6e6e6563
74696f6e3a206b6565702d616c6976650d0a43616368652d436f6e74726f6c3a206d61782d6167653d300d
0a557067726164652d496e7365637572652d52657175657374733a20310d0a557365722d4167656e743a20
4d6f7a696c6c612f352e30202857696e646f7773204e542031302e303b2057696e636343b20783634292041
70706c655765624b69742f3533372e333620284b48544d4c2c206c696b65204765636b6f29204368726f6d
652f38342e302e343134372e313035205361666172692f3533372e33360d0a4163636570743a2074657874
2f68746d6c2c6170706c69636174696f6e2f7868746d6c2b786d6c2c6170706c69636174696f6e2f786d6c
3b713d302e392c696d6167652f776562702c696d6167652f61706e672c2a2f2a3b713d302e382c6170706c
69636174696f6e2f7369676e65642d65786368616e67653b763d62333b713d302e390d0a4163636570742d
456e636f64696e673a20677a69702c206465666c6174650d0a4163636570742d4c616e67756167653a2065
6e2d55532c656e3b713d302e390d0a0d0a
2020-08-10T03:51:47.034166000+0200
2020-08-10T03:51:47.034454000+0200
485454502f312e3120323030204f4b0d0a446174653a204d6f6e2c2031302041756720323032302030313a
35313a3a3520474d540d0a5365727665723a204170616368652f322e342e323920285562756e7475290d0a
4c6173742d4d6f6469666965643a204672692c2030372041756720323032302030303a34353a303220474d
540d0a455461673a202232662d35616333656561346663663031220d0a4163636570742d52616e6765733a
2062797465730d0a436f6e74656e742d4c656e6774683a2034370d0a4b6565702d416c6976653a2074696d
656f75743d352c206d61783d3130300d0a436f6e6e656374696f6e3a204b6565702d416c6976650d0a436f
6e74656e742d547970653a20746578742f68746d6c0d0a0d0a4775722073796e7420766620637670625047
537b633333786e6f30305f315f6633335f685f71726e716f7272737d0a
2020-08-10T03:51:47.074636000+0200
2020-08-10T03:51:52.038759000+0200
2020-08-10T03:51:52.038791000+0200
```

We see a packet with a long hexadecimal payload (the HTTP request) and another with a response payload.

Step 7: Reconstruct and Decode the Payload

Use `tshark` to extract just the payload data, sort by timestamp, convert hex to binary, and display as text:

```
tshark -nr shark1.pcapng -Y "tcp && tcp.stream eq 5" -T fields -e frame.time -e tcp.payload | sort | cut -f2 -d '$\t' | xxd -p -r
```

Output:

```
GET / HTTP/1.1
Host: 18.222.37.134
Connection: keep-alive
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/84.0.4147.105 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9

HTTP/1.1 200 OK
Date: Mon, 10 Aug 2020 01:51:45 GMT
Server: Apache/2.4.29 (Ubuntu)
Last-Modified: Fri, 07 Aug 2020 00:45:02 GMT
ETag: "2f-5ac3eea4fcf01"
Accept-Ranges: bytes
Content-Length: 47
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html

Gur synt vf cvpbPGS{c33xno00_1_f33_h_qrnqorrs}
```

Key finding: The server response contains the string:

```
Gur synt vf cvpbPGS{c33xno00_1_f33_h_qrnqorrs}
```

Step 8: Decode ROT13

The phrase "Gur synt vf" is clearly ROT13 for "The flag is". ROT13 is a simple Caesar cipher that shifts letters by 13 positions.

Decode using `tr` (translate):

```
echo "Gur synt vf cvpbPGS{c33xno00_1_f33_h_qrnqorrs}" | tr 'A-Za-z' 'N-ZA-Mn-za-m'
```

Output:

```
The flag is picoCTF{<REDACTED>}
```

Step 9: Alternative – Decode with CyberChef

You can also decode ROT13 using online tools like [CyberChef](#):

1. Go to <https://gchq.github.io/CyberChef>
 2. Paste the encoded string
 3. Add the "ROT13" operation
 4. The decoded flag appears instantly
-

One-Liner Solution

Here's the complete pipeline to extract and decode the flag in one command:

```
tshark -nr shark1.pcapng -Y "tcp && tcp.stream eq 5" -T fields -e tcp.payload | cut -f2 -d '$'\t' | xxd -p -r | tail -n1 | tr 'A-Za-z' 'N-ZA-Mn-za-m'
```

Final Result

```
picoCTF{<REDACTED>}
```

Note: The actual flag is redacted here. In the real challenge, running the commands above will reveal the complete flag.

Key Concepts

Concept	Description
PCAPNG	Next-generation PCAP format that supports multiple interfaces and enhanced metadata.
TCP Stream	A complete conversation between two endpoints; <code>tcp.stream eq 5</code> filters for a specific stream.
tshark	Command-line version of Wireshark, ideal for automated packet analysis and extraction.
Hex Payload	TCP payloads are often displayed in hexadecimal; <code>xxd -p -r</code> converts hex back to text.
ROT13	A simple substitution cipher shifting letters by 13 positions (self-inverse).
HTTP Traffic	Web traffic contains request headers, response headers, and body content that may hide flags.

Pro Tips

1. **Use `tcp.stream` filters** – This groups packets by conversation, making it easier to reconstruct complete exchanges.
2. **`xxd -p -r` is your friend** – It converts hexadecimal strings back to readable text.
3. **Look for “Gur synt vf”** – This is the classic ROT13 version of “The flag is”. It appears frequently in CTF challenges.
4. **`tr` is perfect for ROT13** – The command `tr 'A-Za-z' 'N-ZA-Mn-a-m'` instantly decodes ROT13.
5. **Check HTTP response bodies** – Flags are often hidden in HTML content or server responses, not just packet headers.

6. **Pipeline power** – Combining `tshark` , `cut` , `xxd` , and `tr` extracts, reconstructs, and decodes in one go.
-

Alternative ROT13 Decoding Tools

Tool	Command / Link
Linux <code>tr</code>	<code>tr 'A-Za-z' 'N-ZA-Mn-za-m'</code>
CyberChef	https://gchq.github.io/CyberChef/ (Operation: ROT13)
Python	<code>import codecs; codecs.decode('text', 'rot_13')</code>
Online	https://rot13.com/